

Phase Vocoder Voice Changer

Independent speed & pitch control for speech, with transient preservation

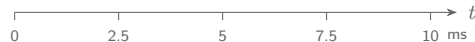
CSE220 — Signals and Systems

Ragib Al-Hasib
Saleh Mustabir Rafi
Afia Jahin Adhitee

Why Not Just Play It Faster?

Sound on a computer is a list of dots, called **samples**. It plays 44 100 dots every second.

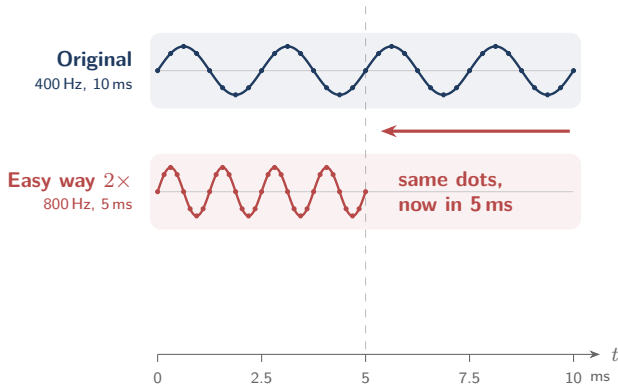
Original
400 Hz, 10 ms



Why Not Just Play It Faster?

Sound on a computer is a list of dots, called **samples**. It plays 44 100 dots every second.

To make it $2\times$ faster, the easy way is to play the dots twice as fast.



Why Not Just Play It Faster?

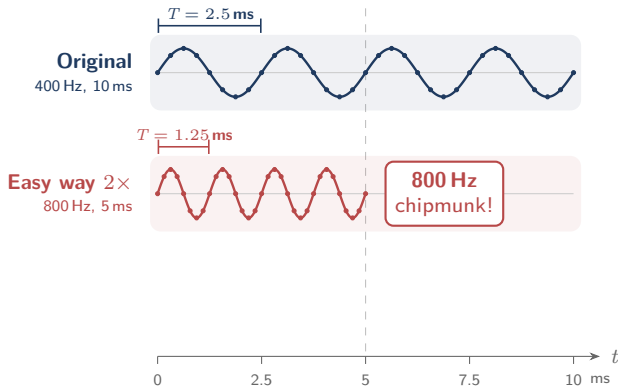
Sound on a computer is a list of dots, called **samples**. It plays 44 100 dots every second.

To make it $2\times$ faster, the easy way is to play the dots twice as fast.

But then every wave gets squeezed.

Squeezed waves sound higher. 400 Hz becomes **800 Hz**. The voice sounds like a **chipmunk**.

Slowing down does the opposite. The voice gets deep and slow.



Why Not Just Play It Faster?

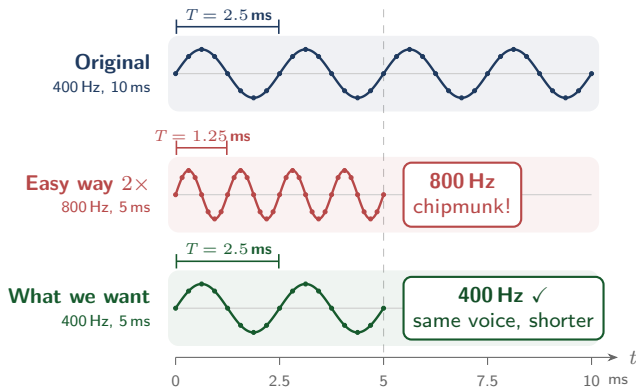
Sound on a computer is a list of dots, called **samples**. It plays 44 100 dots every second.

To make it $2\times$ faster, the easy way is to play the dots twice as fast.

But then every wave gets squeezed.

Squeezed waves sound higher. 400 Hz becomes **800 Hz**. The voice sounds like a **chipmunk**.

Slowing down does the opposite. The voice gets deep and slow.



What we want: a shorter sound with the same waves.

What Goes In, What Comes Out

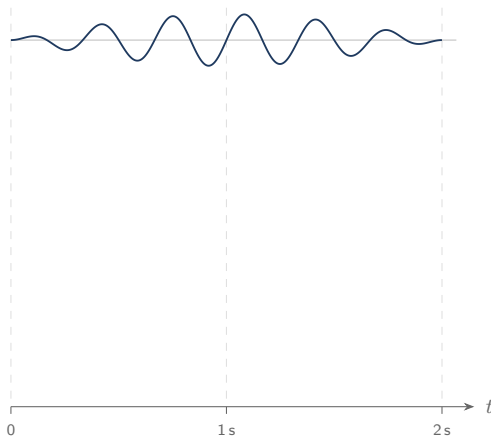
You give it

- a voice clip
- speed: $0.25\times - 2.4\times$
- pitch: up to ± 12 semitones
- mode: Fast or Accurate

You get back

- the new clip
- clear consonants
- graphs to compare
- the chipmunk version (optional)

Your clip
2 seconds



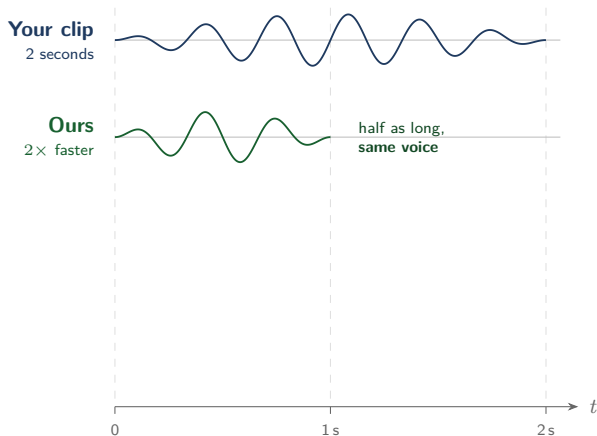
What Goes In, What Comes Out

You give it

- a voice clip
- speed: $0.25\times - 2.4\times$
- pitch: up to ± 12 semitones
- mode: Fast or Accurate

You get back

- the new clip
- clear consonants
- graphs to compare
- the chipmunk version (optional)



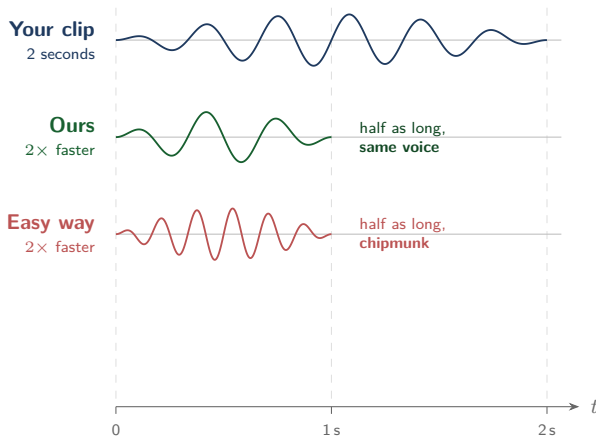
What Goes In, What Comes Out

You give it

- a voice clip
- speed: $0.25\times - 2.4\times$
- pitch: up to ± 12 semitones
- mode: Fast or Accurate

You get back

- the new clip
- clear consonants
- graphs to compare
- the chipmunk version (optional)



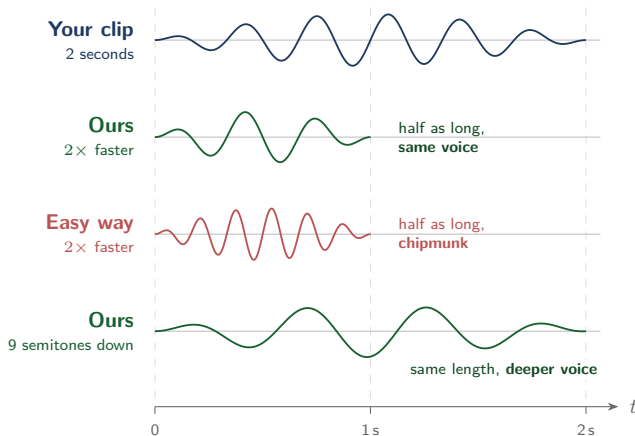
What Goes In, What Comes Out

You give it

- a voice clip
- speed: $0.25\times - 2.4\times$
- pitch: up to ± 12 semitones
- mode: Fast or Accurate

You get back

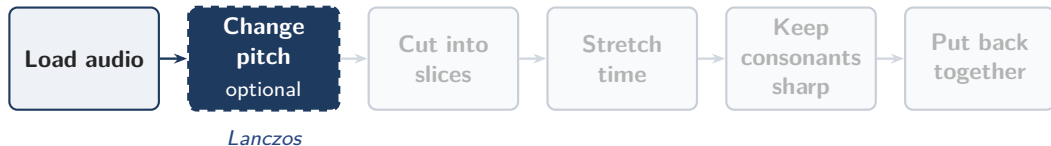
- the new clip
- clear consonants
- graphs to compare
- the chipmunk version (optional)





1. Load the audio

Open a file or record from the mic.
Stereo becomes one track.



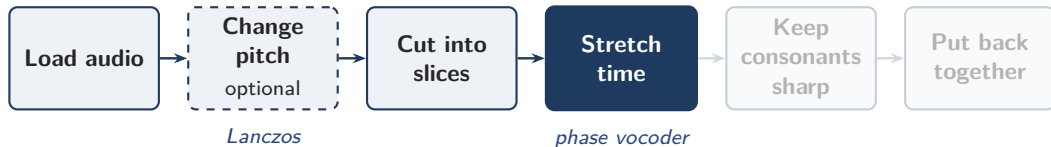
2. Change the pitch (optional)

We play the clip faster or slower. This changes the pitch. It also changes the length. Step 4 fixes the length.



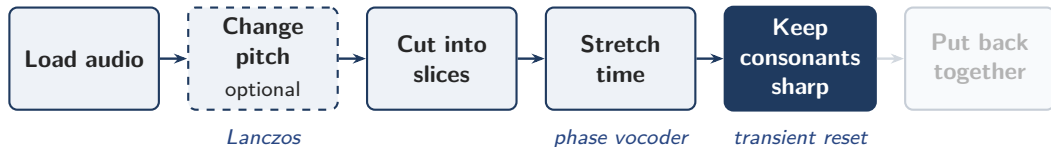
3. Cut into slices

We cut the sound into small pieces that overlap. Each piece is about 46 ms. We check which frequencies are in each piece.



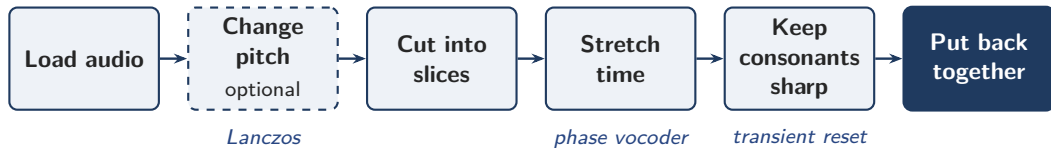
4. Stretch time

To slow down, we spread the pieces apart. To speed up, we push them closer. We keep each frequency lined up, so the pitch stays the same.



5. Keep consonants sharp

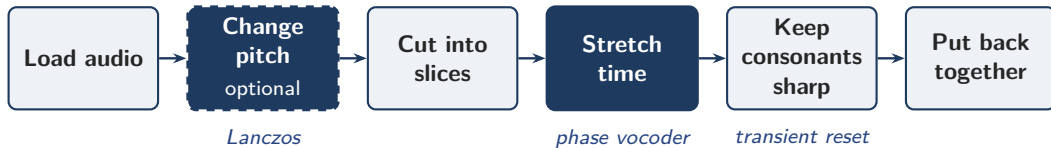
Sounds like “t”, “k” and “p” can get blurry.
We find them and keep them sharp.



6. Put it back together

We turn the pieces back into sound and join them.
Then we fix the volume.

How It Works

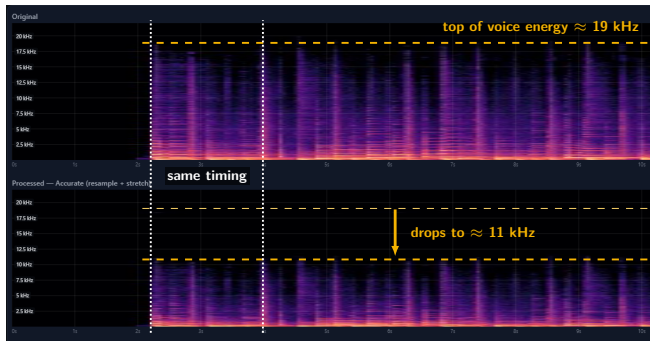


The main idea

Step 2 changes the pitch and the length. Step 4 changes the length back. So only the pitch changes.

Result 1: Deeper Voice, Same Speed

9 semitones down, speed 1×



How to read it

- Left to right: time
- Bottom to top: pitch
- Bright: loud
- Flat stripes: the voice
- Up-and-down lines: consonants

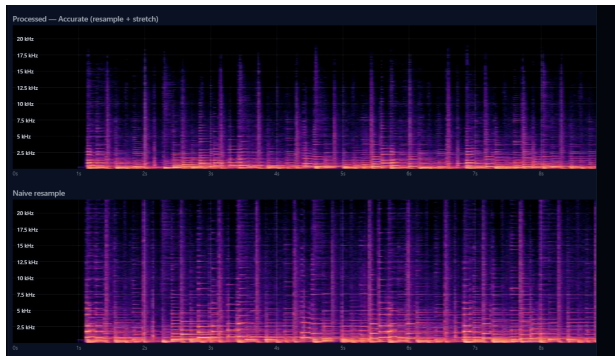
What changed

- The stripes moved down. The voice is deeper.
- The lines stayed in place. The timing is the same.

Pitch changed. Time did not.

Result 2: Faster, Same Voice

speed 2 $\times$, pitch unchanged



Top: ours

Bottom: the easy way

- Both are half as long.
- Easy way: the stripes go up. Chipmunk.
- Ours: the stripes stay. Same voice.

Time changed. Pitch did not.

Choice	Good	Bad
Fast mode	quick	small pitch changes only (± 3 st)
Accurate mode	big pitch changes (± 12 st)	slower
Small slices	sharp timing	blurry pitch
Big slices	clear pitch	blurry timing
<i>For speech, we use a middle size (1024–2048 samples).</i>		
More overlap	smoother sound	slower
Speed limit 0.25–3×	no broken output	extreme settings are blocked

Not done yet: big pitch changes can sound a bit unnatural. Stereo becomes mono.

Let's Hear It



Listen for

1. **Easy way, 2×**
→ chipmunk
2. **Ours, 2×**
→ fast, normal voice
3. **Ours, 9 semitones down**
→ deeper voice, same speed

Speed and pitch, now separate.

Questions?

Ragib Al-Hasib · Saleh Mustabir Rafi · Afia Jahin Adhitee

Backup Slides

for Q&A

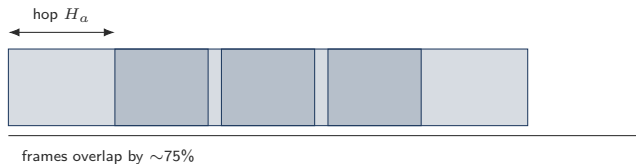
- Reads any decodable file via `soundfile`; multi-channel audio is averaged to mono

$$x_{\text{mono}}[n] = \frac{1}{C} \sum_{c=1}^C x_c[n]$$

- Output: 1-D `float32` in $[-1, 1]$ at the native sample rate
- **Limitation:** stereo width is lost here

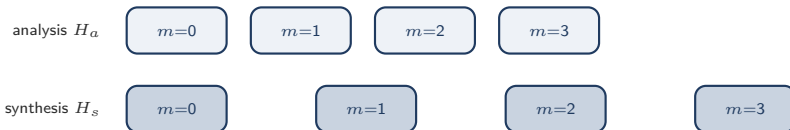
- Cut overlapping frames at hop H_a , multiply by a Hann window, then FFT each frame

$$X[k, m] = \sum_{n=0}^{N-1} x[mR + n] w[n] e^{-j\frac{2\pi}{N}kn}$$



- Estimate each bin's true frequency from its phase drift, then re-accumulate phase at the new hop H_s

$$\hat{\omega}_k = \omega_k + \frac{\Delta\phi - \omega_k H_a}{H_a}, \quad \phi_{\text{out}}[m] = \phi_{\text{out}}[m-1] + \hat{\omega}_k H_s$$

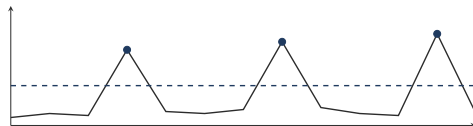


Transient Detection & Phase Reset

- Spectral flux:

$$F_t = \sum_k \max(0, |X_t(k)| - |X_{t-1}(k)|)$$

- Transient if $F_t > \mu_F + 2\sigma_F$, computed over the trailing ~ 250 ms
- On a transient frame, synthesis phase is set to that frame's analysis phase
- A ~ 20 ms refractory window prevents one long attack from re-triggering

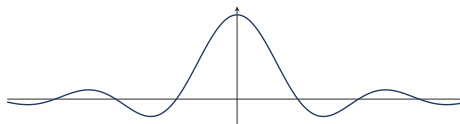


dashed line = adaptive threshold $\mu_F + 2\sigma_F$; dots
= phase-reset frames

- Windowed-sinc interpolation:

$$r = \min(1, s), \quad L_a(x) = r \cdot \text{sinc}(rx) \cdot \text{sinc}(rx/a)$$

- The cutoff lowers when compressing, so it filters while it resamples
- **Why not linear interpolation?** Linear has no anti-aliasing, which causes metallic artifacts



Lanczos kernel, $a = 8$ taps

- IFFT each frame, apply the Hann window, then overlap-add at hop H_s with WOLA normalization

$$y[n] = \frac{\sum_m y_m[n - mR] w[n - mR]}{\sum_m w^2[n - mR]}$$

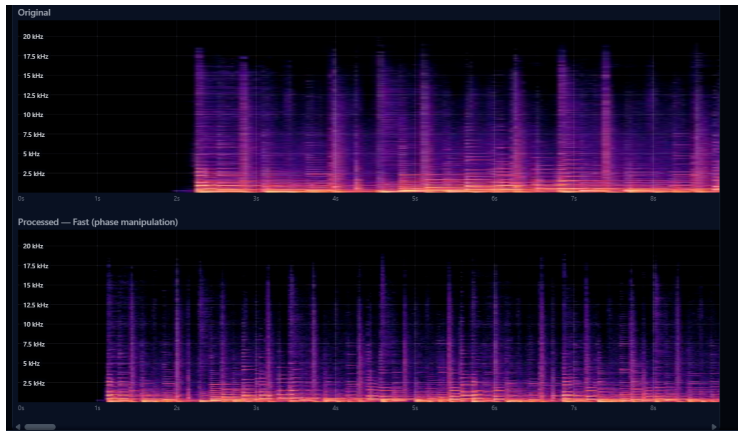
- Peak normalize: $x_{\text{norm}} = 0.99 \cdot x / \max |x|$
- At identity settings, the app reports a round-trip SNR as a sanity check

Window Size & Limits



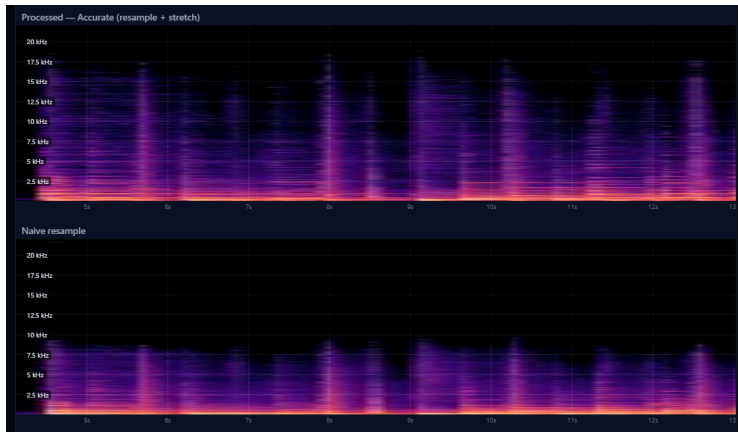
- Speech: 1024–2048 · Music: 2048–4096
- Overlap rises from 75% up to 90% for extreme stretch
- Internal hard limit: compound stretch $0.25\times$ – $3.0\times$. The UI caps speed at $2.4\times$ so that speed combined with pitch stays inside it.
- Pitch artifacts grow past about ± 9 st

Result: Speed-Only 2× (Fast)



Bands stay at the same height while the clip compresses. Speed changed, pitch did not.

Result: Naive vs Accurate, 0.5×



Same slow-down. The naive bands drop; ours stay fixed.

Top: Accurate (resample + stretch) Bottom: Naive resample

- **WOLA normalization:** dividing by the sum of squared windows removes amplitude ripple; with Hann at 75% overlap this sum is nearly constant
- **Phase accumulation:** replaying the analysis phase at a new hop causes warbling, and re-accumulating fixes it
- **Why Hann:** it tapers to zero, which limits spectral leakage. $w[n] = 0.5(1 - \cos(2\pi n/(N - 1)))$
- **Why Lanczos over plain sinc:** smaller sidelobes and sharper rolloff for a similar tap count
- **Threshold choice:** 2σ is a “clearly above noise” heuristic that adapts per frame and excludes the current frame
- **Complexity:** $O(N \log N)$ per frame $\times$ number of frames; the resampler is chunked at 250k samples per chunk
- **Why a hard wall at $0.25\text{--}3.0\times$:** it refuses rather than degrading, because phase-tracking quality is hard to bound near the edges